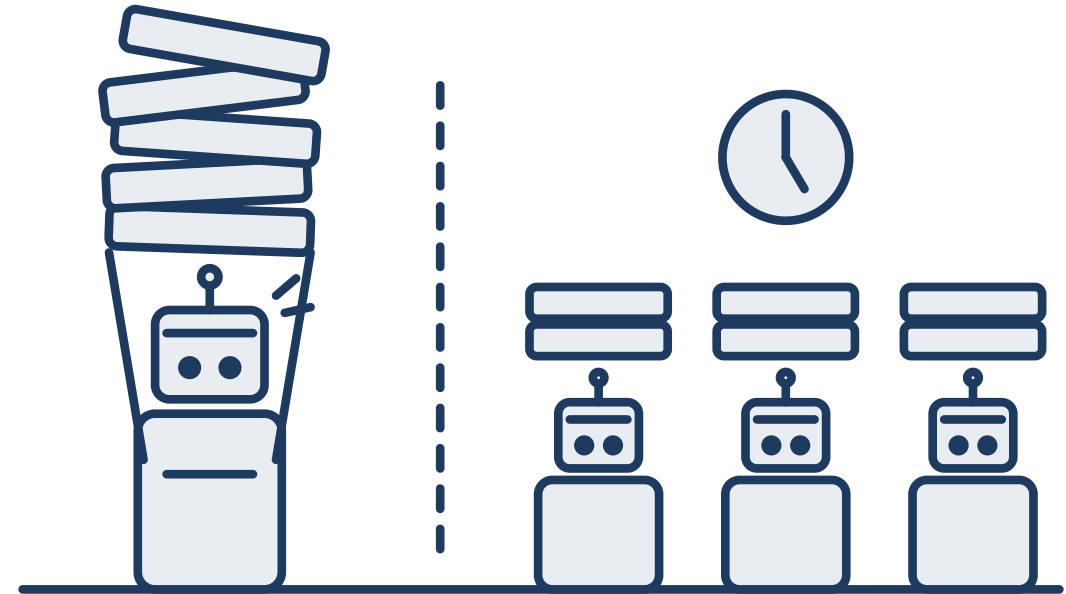


Multi-agent workflows

Shoubaneh Hemmati, Caltech/IPAC.

GRITS AI workshop, September 2026.

Repository: github.com/xoubish/multiGrits



One collaborator, then more

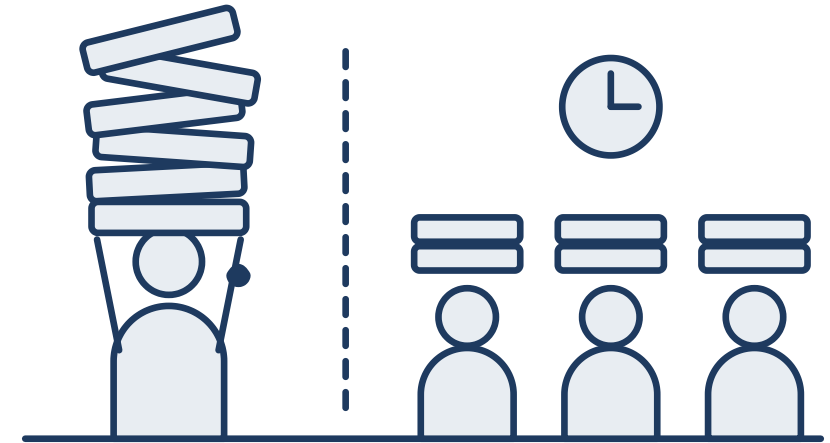
This talk assumes prior use of a coding agent such as Claude Code or Codex on real work. A single agent already plans, writes, checks, and debugs on request, and it has been the best collaborator I have had.

When one collaborator works that well, the obvious next step is a team of them. This talk poses that question before answering it.

Why do people work in teams?

People work in teams for many reasons, some of them social. Four bear on agents, and cost shapes all four.

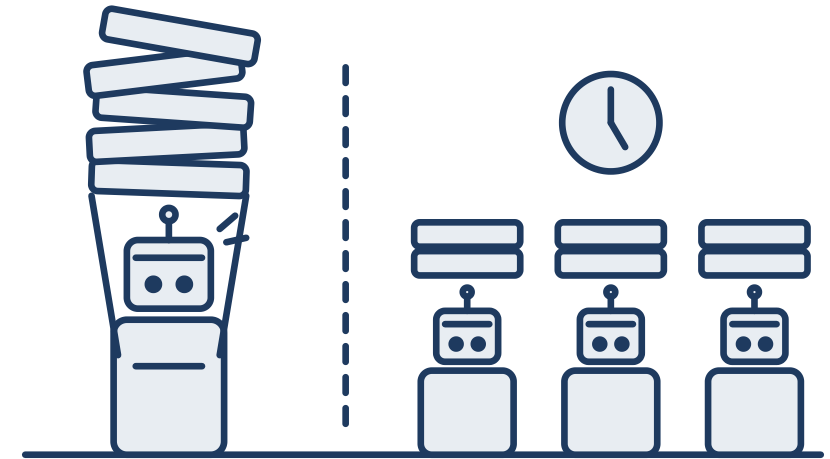
- Time is limited, so the work is divided.
- Knowledge is limited, so expertise is brought in.
- Attention is limited, so pieces are handed off (Simon 1971).
- Judgment is anchored to our own drafts, so others check them.
- Cost sets the mix: one Einstein might do the work of five.



Which of those reasons hold for agents?

Three of the four hold for agents; one holds in part.

- Time holds: agents run in parallel.
- Knowledge holds in part: copies of one model know the same things; mixing models adds some.
- Attention holds: each agent brings a fresh context window.
- Judgment holds: a second agent is not anchored to the first draft.
- Cost sets the mix here too: several cheap models for the price of one frontier model, which often wins anyway.



What multi-agent brings: more context than intelligence

Multi-agent buys more context than intelligence. Windows have grown about five hundredfold in six years; the usable window has not kept pace, and most tasks need none of this.

Year	Model	Context window	Source
2020	GPT-3	2,048 tokens	Brown et al. 2020
2024	GPT-4 Turbo; Claude 3	128K; 200K	Gemini Team 2024
2024	Gemini 1.5	retrieval tested to 10M, in research	Gemini Team 2024
2026	Claude Fable 5.1, Opus 5, Sonnet 5	1M	Anthropic 2026, models page

In English prose a token is about 0.55 to 0.75 of a word, so 1M tokens is roughly 550,000 to 750,000 words (Anthropic 2026).

What fills a context window

- Every turn re-sends the whole window: system prompt, tools, messages, every tool result.
- Tool results dominate: 580K of 613K tokens in this deck's build session.
- When it fills, Claude Code compacts the older transcript into a lossy summary, which drifts as new material is merged in.
- A subagent's transcript stays in its own window; only its return enters the parent's: hierarchical folding, in the literature's term (Hu et al. 2026).

Context usage

claude-fable-5-1 · 613.1k / 1.0M tokens (61%)



Category	Tokens	Share
System prompt	4.9k	0.5%
System tools	23.3k	2.3%
Agents, memory, skills	5.2k	0.5%
Messages and tool results	580.4k	58.0%
Autocompact buffer	33.0k	3.3%
Free space	353.3k	35.3%

Claude Code /context panel, this deck's build session, 2026-09-18. Redrawn; figures as shown.

The usable window is smaller than the advertised one

- Accuracy fell more than twenty points when the answer sat in the middle of the context, below the score with no documents at all (Liu et al. 2023).
- GPT-4 advertised 128K tokens; its effective length was about 64K (Hsieh et al. 2024).
- Models effectively use only 10 to 20 percent of their context (Kuratov et al. 2024).
- Without literal word overlap between question and answer, 11 of 13 long-context models fell below half their short-context accuracy by 32K tokens (Modarressi et al. 2025).
- Human memory shows the same position curve, but the similarity is shallow: people compact to gist within seconds and keep almost no wording, while a model keeps every token and loses the ability to attend to them (Jarvella 1971; Guo and Vosoughi 2025; Hu et al. 2026).

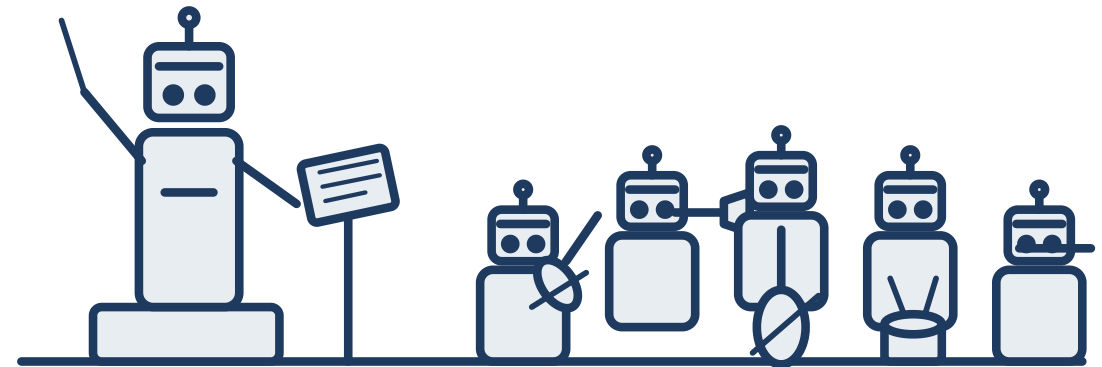
One strong model with a long window

- Given enough budget, one long-context model consistently outperforms retrieval over pieces of the material, at higher cost (Li et al. 2024).
- At an equal thinking budget, a single agent matched or beat five multi-agent designs (Tran and Kiela 2026).
- A larger window does not remove the problem: as history accumulates, attention saturates and the goal drifts; the bottleneck is maintaining task state, not capacity (Hu et al. 2026).
- Splitting pays only when the work exceeds what one strong window handles well, or when independence or wall-clock matter.

Elements of a multi-agent system

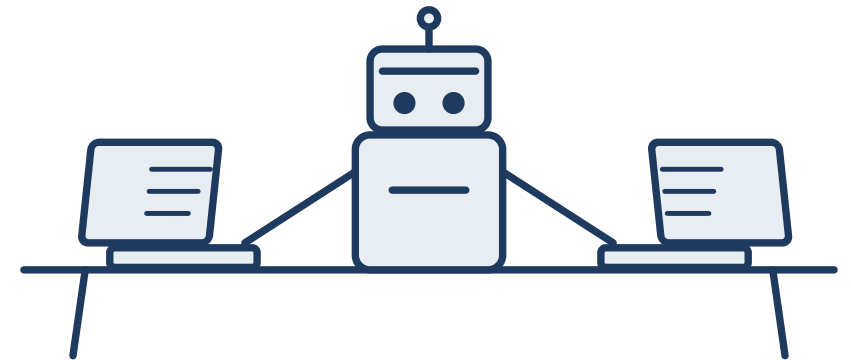
Designing a multi-agent system means thinking along three axes, taken one at a time on the next three slides.

- **Who orchestrates:** a script, a model, or a human.
- **Topology:** how the agents are connected.
- **Isolation:** shared or fresh context; shared files or separate git worktrees.



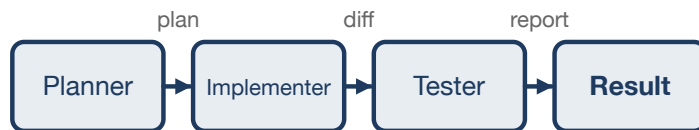
Axis 1, who orchestrates: a script, a model, or a human

- A script fixes the sequence in advance: which agent runs, in what order, how many rounds. Anthropic calls this a workflow.
- The model chooses each next step at run time, including whether to call another agent: an agent (Anthropic 2024).
- A person can orchestrate too. Scripts are reproducible; a model deciding is flexible but hard to reproduce.
- At the frontier the orchestrator is itself learned: searched workflows have beaten human-designed ones, at a cost per candidate (Gao et al. 2026).

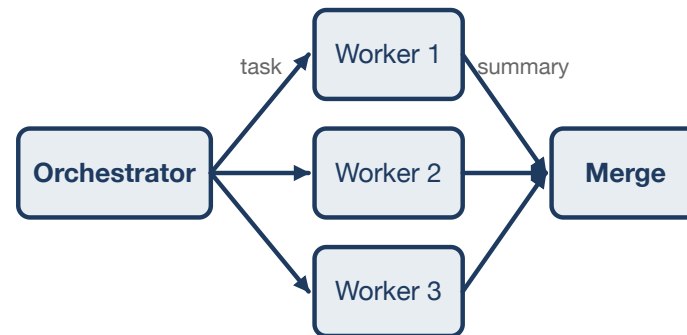


Axis 2, topology: how the agents are connected

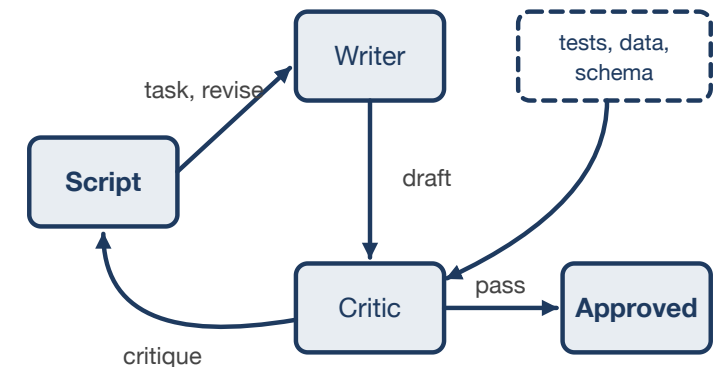
Three shapes cover most systems. Errors are caught at the merge point, or not at all (Kim et al. 2026).



Chain: each stage hands its output to the next, files as the hand-off. A planner writes the plan, an implementer the code, a tester runs it.



Fan-out and merge: independent pieces, one merge point. Each worker returns a summary, not its raw material.



Loop: writer and critic pass work back and forth a fixed number of times. The critic holds the tests and data; a critic that can only read prose agrees with it.

Axis 3, isolation: what the agents share

- Context: shared, so one agent carries everything; or fresh, so each stage starts empty and receives only what it needs. Fresh context is the gain argued for above.
- Files: shared, so two agents can write the same file; or one git worktree per agent, merged at the end.
- Neither is free. Isolated memories bring redundancy and communication overhead; shared stores bring clutter and write contention (Hu et al. 2026).
- A Claude Code subagent starts with a fresh context and receives only the delegation message (Anthropic 2026).
- Write contention is the failure in attempt one: two agents, one file.

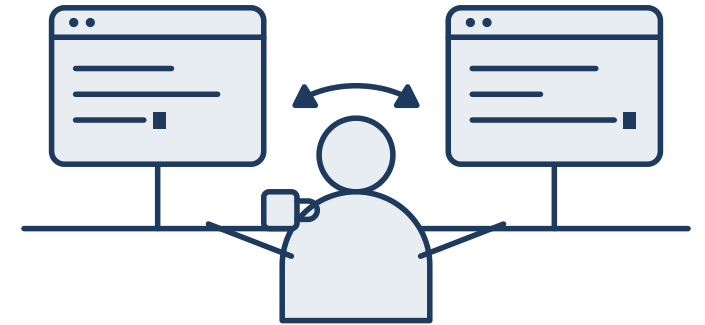
The more the merrier?

Scale	What it is	Who
1	The default. One conversation; a subagent is spawned only when the model decides to (Anthropic 2026).	Everyday use
2 to 7	One orchestrator and a few workers: a lead plus subagents (Anthropic 2025), five fixed roles (Hong et al. 2024), seven (Qian et al. 2024); three to five teammates is the vendor's own guidance (Anthropic 2026).	Working systems
Tens	Sampling and voting keeps improving into the tens of samples, but 40 samples of a weak model still trail one call of a strong one (Li et al. 2024), returns diminish once the single agent is strong (Kim et al. 2026), and agents grow reliant on group consensus and lose independent reasoning (Gao et al. 2026).	Benchmarks
1,000+	Collaboration among over a thousand agents (Qian et al. 2024); simulations of 10 to 1,000+ (Altera 2024). Nobody uses this for work.	Research

There is no published survey of what practitioners run; the "Who" column is an assessment.

You already do this

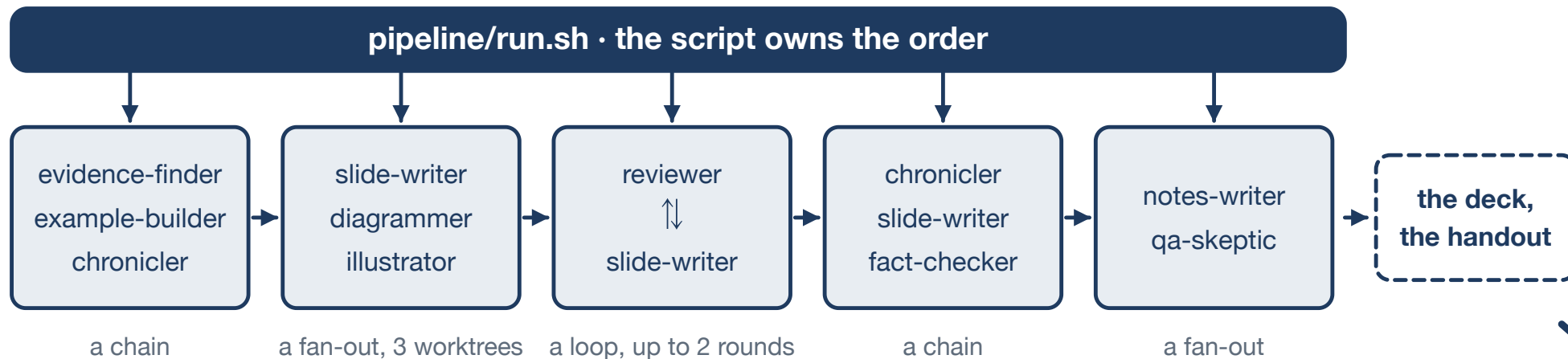
- Ask ChatGPT a question, paste the answer into Claude, and ask whether it is right. **You orchestrate: writer and critic.**
- Five Claude Code windows, each writing the code for one plot; a sixth adds them, with text, to the paper. **You orchestrate: fan-out and merge.**
- A colleague's Claude Code writes a tutorial notebook and delivers a git repo; a second colleague's reviews it and delivers comments on a new branch; yours applies them and opens a pull request. **Your boss orchestrates: a pipeline.**



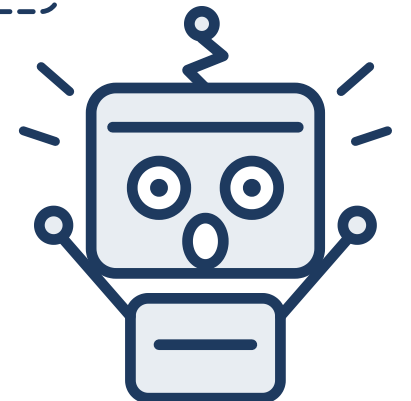
When to make it deliberate, and hand the orchestration to a script or a model?

This deck was built by a workflow in this repo

Ten agents drafted every part of this deck: the slides, the diagrams, the evidence, the notes and the Q&A. A shell script called them in a fixed order and logged every call.



What follows is how it was built, and then what went wrong with it.



What I gave it: one spec file

Not a prompt. `talk-context.md`, 122 lines in nine sections, read by every agent before its own instructions.

```
## The event          speaker, venue, the binding 27 + 3 minute budget
## Audience          who they are and what they already know
## Full schedule      what every neighbouring session covers
## Explicit non-goals what this talk must not duplicate
## Thesis            more context than intelligence
## Structure          the outline is the spec; no agent may edit it
## What an attendee can do afterwards
## The meta-example   this deck is built by the pipeline it describes
## Style constraints  full sentences, compact citations, one idea per slide
```

The decisions, on the three axes

- **Who orchestrates.** Claude Code itself, interactively, with me steering. It then wrote the shell script that replaced it, and the script has run every stage since.
- **Topology.** A chain: each stage finishes before the next begins. Two of the stages fan out to several agents at once, and one is a loop between a writer and a reviewer.
- **Isolation.** Every agent starts with an empty conversation and remembers nothing from the last one. The three that run at the same time also get their own copy of the repository, so they cannot overwrite each other's files.

None of it needed a framework: three decisions, a few lines of shell.

Recipe: an agent is a markdown file

This is `.claude/agents/reviewer.md`, trimmed to fit. The frontmatter names the agent, says when to use it, lists its tools, and picks the model.

```
---  
name: reviewer  
description: Sits in the audience. Reads the deck as a skeptical IPAC engineer  
             and looks at the rendered slide images. [...]  
tools: Read, Glob, Grep, Write  
model: inherit  
---
```

Recipe: the instructions

Below the frontmatter come instructions in plain English, like a brief to a colleague. The last line matters, because the script parses it to decide whether to loop.

```
You are the one reviewer. You replace three earlier critics whose rubrics counted citations, words per second, and seconds per slide; the deck they passed was unpresentable. Do not count things. Judge whether the talk works.
```

```
5. Is there a moment where the speaker admits something did not work? If not, say where one belongs.
```

```
Write to the run directory you are given as `review.md`. First line exactly `Verdict: PASS` or `Verdict: REVISE`.
```

Recipe: what the orchestrator wrote

Ten agent files, and 418 lines of shell and Python underneath them: the runner, the cost logger, the renderer, the diagram inliner.

talk-context.md	the spec above, read by every agent
slides/outline.md	HUMAN-WRITTEN. Slide order, message, time budgets. No agent edits it.
.claude/agents/	ten agent definitions, one markdown file each
pipeline/run.sh	203 lines: stages, the review loop, worktree isolation
pipeline/*.py	cost report, result logger, diagram inliner, renderer
research/evidence.md	one source per claim in the outline
runs/	one directory per agent call: prompt, JSON result, return, cost

Recipe: one headless call per agent

`--agent` picks the markdown file, `--output-format json` returns a parseable result with the cost in it, `--max-budget-usd` caps the spend, and `--no-session-persistence` means no state carries between calls.

```
env -u CLAUDECODE claude -p --agent "$agent" \  
  --output-format json \  
  --permission-mode acceptEdits \  
  --allowedTools "$tools" \  
  --max-budget-usd "$BUDGET" \  
  --no-session-persistence \  
  "$(cat "$run_dir/prompt.md")" > "$run_dir/result.json"
```

Recipe: the stage order

```
evidence → example → chronicle → write (slide-writer + diagrammer + illustrator, 3 worktrees, merged)
  → loop (critique → revise, until PASS or 2 rounds) → chronicle → revise → factcheck
  → notes & qa (parallel) → cost
```

Evidence, example and chronicle run one after another, which is a chain. Write is a fan-out into three worktrees. The loop is writer and critic, and the script owns the loop, not the agents. Cost is a Python script and uses no model at all.

Attempt one, from scratch

The first pipeline let the agents write the outline as well as the slides. Eleven agent files did the work: four researchers in parallel, an outliner, a slide-writer and a diagrammer in worktrees, three critics for content, design and teaching, a fact-checker, a notes-writer, a Q&A skeptic and a demo editor.

Runs 002 through 014 cost \$29.22 over 455 turns. Three things broke.

Attempt one, failure 1: the spec contradiction

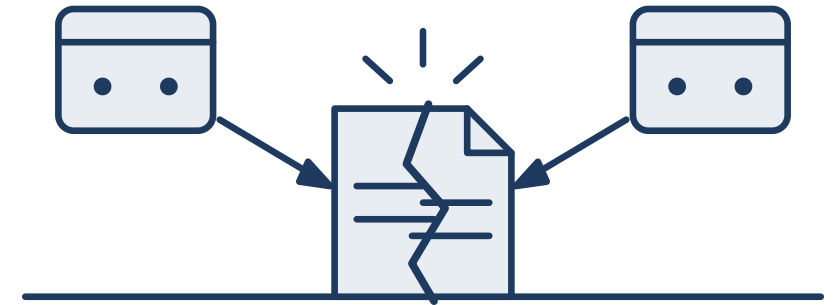
In run 002, the outliner found that my segment table summed to 27 minutes of content plus 3 of Q&A, my prose said 25 plus 5, and its own instructions demanded 1500 seconds. It could not satisfy all three, so it picked one, scaled three segments down, and logged the choice in its notes rather than choosing silently.

Fix: the spec, not the agent. The stage was re-run as run 003.

Attempt one, failure 2: the shared cost log

In run 004, the slide-writer and the diagrammer ran in two worktrees and never touched each other's outputs. But `log_result.py` appended one row to a single shared `runs/cost.tsv` from inside each worktree, so the merge conflicted on the pipeline's own bookkeeping.

Fix: each run directory writes its own `cost-row.tsv`, and the report is regenerated from every `result.json`.



Attempt one, failure 3: the budget cap

In run 006, the first revise pass exhausted the then-default \$3 `--max-budget-usd` ceiling after 34 turns. `result.json` said `terminal_reason: budget_exhausted`, but the exit code was 1, not the 2 the docs describe. It had already applied 20 of its 24 changes.

Fix: the default budget went to \$5, and the script reads `result.json` instead of trusting the exit code.

What came out

Every constraint was met: citations complete, word counts under the cap, segment timing to the second. The fact-checker confirmed 44 citations, 2 partial, 0 not found.

And it was a wall of cited percentages, plus a twelve-minute demo of screenshots of README files. This is one of its slides.

Gotcha · Isolate the files, centralize the merge

Two agents editing one file is the classic failure. Worktrees fix it [Claude Code worktrees].

Independent agents with no central check amplified errors 17.2x; with centralized verification, 4.4x [Kim 2026].

"The bottleneck is no longer generation, it's verification" [Osmani].

What came out, and why

The three critics scored what could be counted: citation counts, words per second, and seconds per slide. Nobody scored whether it was a talk.

Agents optimize the rubric they are given. Taste is not in the rubric, and it cannot be, so taste has to be a human's.

Why multi-agent: context, not intelligence

- Answer placed mid-context: accuracy drops over 20 points, below the no-documents score [Liu 2023]
- GPT-4 claimed 128K tokens; effective 64K on a long-context test [Hsieh 2024]
- Fix: delegate bounded subtasks; each subagent returns a 1,000–2,000-token summary [Anthropic 2025a]

The reset: the outline became mine

I wrote this outline by hand, and no agent generates or reorders it. This is its first entry exactly as it sits in `slides/outline.md`; every entry has that shape:

```
1. Title. Multi-agent workflows. Shoubaneh Hemmati, Caltech/IPAC. GRITS AI workshop, September 2026. 15 s.
```

Agents draft slides, diagrams, evidence, notes and reviews for that outline. If one finds a structural problem, it reports it under a "For the speaker" heading and does not fix it.

The reset: retired and added

Retired. The outliner, because I own the outline. The four researchers, whose push-mode briefs nobody used. The three critics, three rubrics that rewarded compliance. The demo editor.

Added. An evidence-finder that sources only the claims I make. An example-builder that runs the take-home example for real. One reviewer that sits in the audience. A chronicler that records the build. An illustrator.

The reset: research, pushed then pulled

- **Pushed.** Four researchers wrote about 7,100 words of briefs on four topics I had guessed at in advance. Most of it was never used.
- **Pulled.** One evidence-finder reads the finished outline and finds a source for each claim I actually make.
- It sourced 13 of 16 claims from the old briefs and flagged 4 as unsupported rather than guessing at them.

Research against the claims, not against the subject.

Attempt two, run by run

Runs	Stage	What happened	Cost
015–017	evidence, example, chronicle	Sourced 13 of 16 claims; built and ran the exoplanet example; wrote the build log.	\$1.49
018	write	Slide-writer, diagrammer and illustrator in three worktrees at once, merged with no conflict.	\$4.19
019–022	critique, revise, twice	Both rounds returned REVISE: illustrations unplaced, code at 7 pixels, diagrams squeezed.	\$9.10
023–027	chronicle, revise, fact-check, notes, Q&A	The log caught up with the build, every source was verified, the handout was written.	\$5.19
029–030	revise, notes	An editorial review written by hand, not by the reviewer agent, made the posted slides stand alone.	\$5.12

What it cost, and the receipts

- Attempt one: \$29.22 over 455 turns. Attempt two: \$25.09 over 563. Both: **\$54.31** over 1,018 turns.
- Every call left a directory holding the exact prompt sent, the full JSON result, the return text, the exit code and its own cost row.
- The report is regenerated from every `result.json`, which is how a pipeline is audited rather than trusted.
- The field's own deployment checklist asks for exactly this: an immutable audit trail and version-controlled safe states (Gao et al. 2026).

One agent, one slide at a time

- The reviewer reads the deck and looks at every rendered slide image, then writes `Verdict:` `PASS` or `Verdict: REVISE` .
- The slide-writer applies one review at a time and logs each finding it declined, and why.
- A review I write by hand goes in the same file, in the same format. The script cannot tell the difference; run 028 was one of mine.
- That is where this deck is now. It has not returned PASS.

The one paper that says both

Kim and colleagues ran the same experiment both ways, over 260 configurations, 6 benchmarks and 3 model families.

- A decomposable financial-reasoning task improved by 80.8 percent with a centralized multi-agent design (Kim et al. 2026).
- A sequential planning task got 39 to 70 percent worse, under every multi-agent variant they tried (Kim et al. 2026).
- The inference is mine, not theirs: work that splits into independent pieces is a candidate for fan-out, and work where each step depends on the last belongs in one window.

The rest of the evidence, both sides

- **In favor.** A lead agent plus subagents did 90.2 percent better than a single agent on breadth research, at about 15 times the tokens of a chat. A vendor claim, not independently replicated (Anthropic 2025).
- **Against.** At an equal thinking budget a single agent matched or beat five multi-agent designs, 0.427 to 0.386. Multi-agent won only when up to 70 percent of the context was masked (Tran and Kiela 2026).
- **Against.** One agent framework cost over 50 times a simple baseline, and the baseline was more accurate, 93.2 percent to 88.0 (Kapoor et al. 2024).

The honest summary is that it depends on the task, and the token multiple is real either way.

How it fails

The MAST study annotated over 1,600 traces across seven frameworks and found fourteen failure modes in three categories (Cemri et al. 2025).

- System design and specification: 44.2 percent.
- Inter-agent misalignment: 32.3 percent.
- Task verification: 23.5 percent.

Most failures are specification and coordination problems, not model errors.

How it fails in practice

First, subagents do not see the orchestrator's conversation. They must be given what they need in the prompt, and asked for a summary in return, not a dump.

Second, two agents editing one file; the shared cost log in attempt one is an instance.

Third, agents spawning agents instead of a script calling agents. Once the model decides the order, reproducibility is lost.

A simple example to run first

The example is one read-only subagent. Given ten target names, it queries the NASA Exoplanet Archive with astroquery and returns a table under a token cap. It has two tools, Read and Bash, and the cheapest model. The description is cut short with an ellipsis; the full file is in `examples/`.

```
---
name: exoplanet-lookup
description: Read-only lookup of confirmed exoplanet parameters from the
  NASA Exoplanet Archive for a given list of planet names. [...]
tools: Read, Bash
model: haiku
---
```

The example: the instructions

Below the frontmatter are three rules in plain English: read the file, query the archive, and say "not found" rather than invent a number.

You are given a path to a text file with one exoplanet name per line (up to ten names).

Do this:

1. Read the file.
2. For each name, query the NASA Exoplanet Archive (table `pscomppars`) with astroquery for columns: `pl\_name, hostname, pl\_orbper, pl\_rade, pl\_bmasse, disc\_year`. One Python process, one query per name (or a single `where` clause with all names), is fine.
3. If a name has no match, say "not found" in that row. Do not invent numbers.

The example: the return format

The return format follows. This is the part that keeps the subagent from dumping its whole session back into the orchestrator's window: one table, a source line, and a token cap.

Return only a single compact markdown table, one row per input name, columns:

``name | host | period_days | radius_earth | mass_earth | disc_year``.

Round numbers to 3 significant figures. No prose before or after the table, except a one-line header stating the source ("NASA Exoplanet Archive, pscomppars"). Keep the whole reply under 600 tokens.

The example: the command

The call is one command, headless, with a one-dollar cap. The output is JSON, so the cost and the token counts come back with the answer.

```
env -u CLAUDECODE claude -p --agent exoplanet-lookup --allowedTools "Read,Bash" \  
  --max-budget-usd 1 --output-format json \  
  "Look up the exoplanets listed one per line in targets.txt in the NASA Exoplanet Archive and return the table."
```

The example: the real result

NASA Exoplanet Archive, pscomppars

name	host	period_days	radius_earth	mass_earth	disc_year
Kepler-10 b	Kepler-10	0.837	1.47	3.24	2011
TRAPPIST-1 e	TRAPPIST-1	6.10	0.920	0.692	2017
Proxima Cen b	Proxima Cen	11.2	1.02	1.05	2016

The reply was ten rows and about 250 tokens. The model was `claude-haiku-4-5`, the cost \$0.0751, the wall time 82.5 seconds over 8 turns. One caveat: total output including thinking was 7,321 tokens, so the cap applies to the table returned, not to everything the agent emits.

Cost, and when it is worth it

- A multi-agent system uses about 15 times a chat's tokens (Anthropic 2025).
- Later guidance from the same vendor says 3 to 10 times a single agent's (Anthropic 2026).
- Both figures come from the party promoting multi-agent, which has no reason to overstate the cost.

Spending the multiple

- Cheap or local models belong on the subagents, and the frontier model on the orchestrator.
- A subagent's frontmatter takes a `model` field, so the choice is one line per agent (Anthropic 2026).
- In this pipeline the reviewer and the slide-writer run on the frontier model; everything else runs on the cheaper one.

Model selection and pricing are covered elsewhere in the workshop and are not repeated here.

The difficult example

The difficult example is this pipeline: ten agents, a script that owns order, loop and isolation, and a log for every call.

It is worth it when all four hold:

- The work exceeds one context window.
- The pieces are independent of each other.
- The output needs a check from something that did not write it.
- The token multiple is affordable.

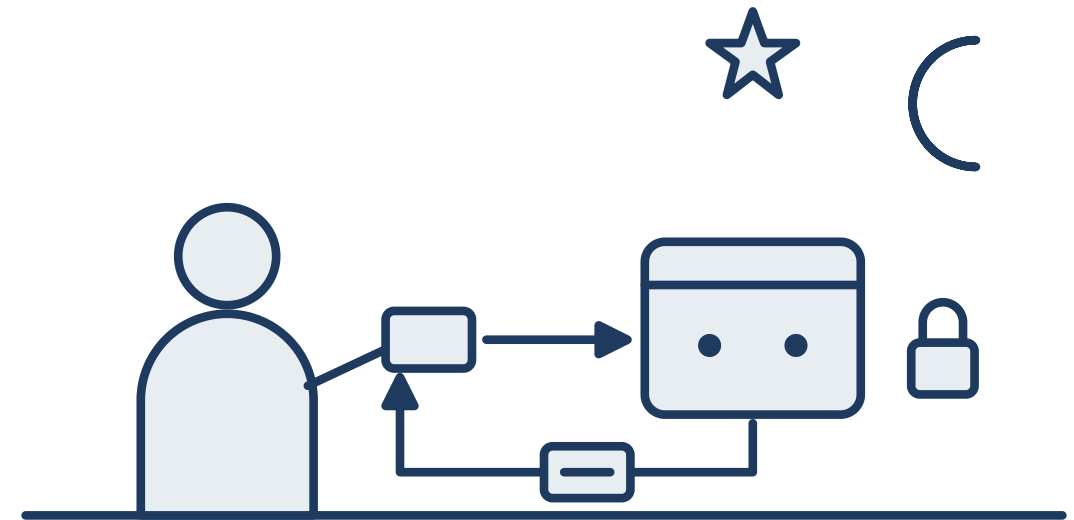
Otherwise one agent suffices. Most of my own work is still one agent.

Close

More context than intelligence. Most tasks need one agent.

A first step is to pull one bounded, read-only task into its own agent file, run it once headless with a budget cap, and check that the return is short.

One closing line: unsupervised agents require sandboxing and approval gates, the first items on the field's own deployment checklist (Gao et al. 2026), and the subject of the following session.



Sources: segments A and B

Simon, H. A. (1971), Designing Organizations for an Information-Rich World, in M. Greenberger (ed.), Computers, Communications, and the Public Interest, Johns Hopkins Press, pp. 37–72.

Brown et al. (2020), Language Models are Few-Shot Learners: <https://arxiv.org/abs/2005.14165>

Gemini Team (2024), Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context: <https://arxiv.org/abs/2403.05530>

Anthropic (2026), Models overview, Claude Platform docs: <https://platform.claude.com/docs/en/about-claude/models>

Liu et al. (2023), Lost in the Middle: <https://arxiv.org/abs/2307.03172>

Hsieh et al. (2024), RULER: <https://arxiv.org/abs/2404.06654>

Tran et al. (2025), Multi-Agent Collaboration Mechanisms: A Survey of LLMs: <https://arxiv.org/abs/2501.06322>

Kuratov et al. (2024), BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack: <https://arxiv.org/abs/2406.10149>

Modarressi et al. (2025), NoLiMa: Long-Context Evaluation Beyond Literal Matching: <https://arxiv.org/abs/2502.05167>

Jarvella, R. J. (1971), Syntactic processing of connected speech, Journal of Verbal Learning and Verbal Behavior 10, 409–416.

Guo and Vosoughi (2025), Serial Position Effects of Large Language Models, Findings of ACL 2025: <https://arxiv.org/abs/2406.15981>

Hu et al. (2026), Memory in the Age of AI Agents: A Survey, Forms, Functions and Dynamics: <https://arxiv.org/abs/2512.13564>

Gao et al. (2026), A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve, TMLR: <https://arxiv.org/abs/2507.21046>

Li et al. (2024), Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach: <https://arxiv.org/abs/2407.16833>

Anthropic (December 2024), Building effective agents: <https://www.anthropic.com/research/building-effective-agents>

Anthropic (2026), Run parallel sessions with worktrees, Claude Code docs: <https://code.claude.com/docs/en/worktrees>

Anthropic (2026), Subagents, Claude Code docs: <https://code.claude.com/docs/en/sub-agents>

Anthropic (June 2025), How we built our multi-agent research system: <https://www.anthropic.com/engineering/multi-agent-research-system>

Sources: segments C, D, and E

Segment C has no external sources: research/build-log.md, README.md, pipeline/run.sh, .claude/agents/reviewer.md, examples/exoplanet-lookup/, and runs/ in this repository: <https://github.com/xoubish/multiGrits>. The first deck is at commit 707ad48.

Kim et al. (2026), Towards a Science of Scaling Agent Systems: <https://arxiv.org/abs/2512.08296>

Anthropic (June 2025), How we built our multi-agent research system: <https://www.anthropic.com/engineering/multi-agent-research-system>

Tran and Kiela (April 2026), Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets: <https://arxiv.org/abs/2604.02460>

Kapoor et al. (2024), AI Agents That Matter: <https://arxiv.org/abs/2407.01502>

Cemri et al. (2025), Why Do Multi-Agent LLM Systems Fail? (MAST): <https://arxiv.org/abs/2503.13657>

Gao et al. (2026), A Survey of Self-Evolving Agents, TMLR (deployment checklist, Table 12): <https://arxiv.org/abs/2507.21046>

Anthropic, Phillips et al. (January 2026), Building multi-agent systems: when and how to use them: <https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them>

Anthropic (2026), Create custom subagents, Claude Code docs: <https://code.claude.com/docs/en/sub-agents>